

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(МАГИСТЕРСКАЯ РАБОТА)
по направлению подготовки
09.04.01 - «Информатика и вычислительная техника»**

**GRADUATION THESIS
(MASTER'S GRADUATION THESIS)**

**Field of Study
09.04.01 – «Software Engineering»**

**Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Software Engineering»**

**Тема /
Topic**

**Анализ объектных моделей в современных языках
программирования/
Analysis of object models in modern programming languages**

**Работу выполнил /
Thesis is executed by**

**Шалагин Егор Сергеевич /
Shalagin Egor**

подпись / signature

**Руководитель
выпускной
квалификационной
работы /
Supervisor of
Graduation Thesis**

**Зуев Евгений
Александрович /
Zouev Eugene**

подпись / signature

**Консультанты /
Consultants**

подпись / signature

Иннополис, Innopolis, 2025

Contents

1	Introduction	7
2	Literature Review	9
2.1	The Concept and Components of the Object Model	10
2.1.1	Major Components of the Object Model	11
2.1.2	Minor Components of the Object Model	14
2.1.3	The Selected Definition of the Object Model	16
2.2	Critical Analysis of Object-Oriented Programming Approach . .	17
2.2.1	Theoretical Inconsistencies in Inheritance and Subtyping	17
2.2.2	The Fragile Base Class Problem	18
2.2.3	Excessive Coupling and Tight Interdependencies	20
2.2.4	Multiple Inheritance and Ambiguity Resolution	28
2.2.5	The Reusability Paradox	31
2.2.6	State Management Complexity and Object Identity . . .	32
2.2.7	Absence of Clear Separation Between Specification and Implementation	37
2.2.8	Empirical Evidence of Design Difficulties	37
2.2.9	Systematic Classification of OOP Limitations	39
2.3	Review of Related Work	44

2.3.1	Fundamental taxonomies and theoretical foundations . .	44
2.3.2	Empirical Performance Comparisons	45
2.3.3	Narrowly Focused and Engineering Comparisons	45
2.3.4	Conclusion	46
2.4	Methodology of Comparative Analysis	47
2.4.1	Rationale for the Chosen Methodology	47
2.4.2	Object Model Comparison Criteria	48
2.4.3	Methodology of Comparative Analysis	50
3	Methodology	54
3.1	Comparison Technique	55
3.1.1	Type System and Subtyping Model	55
3.1.2	Abstraction and Encapsulation Mechanisms	57
3.1.3	Inheritance Semantics and Hierarchy Management . . .	59
3.1.4	Composition and Alternatives to Inheritance	61
3.1.5	Object Representation and Creation Model	63
3.1.6	Polymorphism and Dispatch Mechanisms	65
3.1.7	State Model, Mutability, and Concurrency Guarantees .	67
3.1.8	Support for Formal Specification and Verifiability	69
4	Implementation	71
5	Evaluation and Discussion	77
6	Conclusion	78
	Bibliography cited	79

List of Tables

List of Figures

2.1	Inheritance without proper subtyping	19
2.2	Inheritance without proper subtyping: manifestation	20
2.3	Fragile Base Class: first implementation base class	21
2.4	Fragile Base Class: first implementation deriving class	22
2.5	Fragile Base Class: misspelled implementation base class	23
2.6	Fragile Base Class: misspelled implementation deriving class	24
2.7	Fragile Base Class: demonstration	24
2.8	Base class Account exposing internal state and workflow through protected members	25
2.9	Derived class SavingsAccount tightly coupled to parent's imple- mentation details	26
2.10	Class Order establishing bidirectional dependency with Customer	26
2.11	Class Customer completing circular dependency cycle and de- pending on Order internals	27
2.12	Supporting class OrderItem whose internal calculation logic is exposed to dependent classes	27
2.13	Base class Animal defining shared state and behavior in diamond inheritance hierarchy	29

2.14	Intermediate class Mammal inheriting from Animal without overriding base methods	29
2.15	Intermediate class Bird inheriting from Animal without overriding base methods	29
2.16	Derived class Bat exhibiting diamond inheritance ambiguity without virtual inheritance	29
2.17	Base class Animal defining shared state in Python diamond hierarchy	30
2.18	Intermediate class Mammal inheriting from Animal without method override	30
2.19	Intermediate class Bird inheriting from Animal without method override	30
2.20	Derived class Bat demonstrating method resolution order limitations in Python	30
2.21	Instrumented set implementation using inheritance	32
2.22	Instrumented set implementation using composition	33
2.23	Mutable entity class with identity and state-based equality semantics	35
2.24	Demonstration of identity-value tension and state mutation effects	36
2.25	Base class implementing the Template Method pattern for order processing.	38
2.26	Inheritance layer introducing payment and transaction persistence behavior.	39
2.27	Inheritance layer adding fraud detection and blacklist management.	40
2.28	Inheritance layer introducing retry semantics around the processing algorithm.	40
2.29	Concrete enterprise processor exhibiting extensive infrastructure coupling.	43

Abstract

This work conducts a systematic comparison of the object models in ten modern programming languages: C#, C++, Golang, Java, Python, Ruby, JavaScript, Scala, Smalltalk, and Zonnon. The research is motivated by the operational challenges faced by software engineers when choosing technologies given the significant differences in how object models are implemented in different language ecosystems. The main objective of the research is to identify the similarities, differences, strengths, and weaknesses of various object models, and based on this analysis, to develop proposals for an enhanced model. As a result of the study, the theoretical foundations of models are systematized, a detailed examination of their implementation in the selected languages is conducted, identifying application domains and common pitfalls, and an original model integrating best practices is proposed. The results obtained serve as a practical guide for developers when choosing a language and for software architects when designing object-oriented systems.

Chapter 1

Introduction

Object-oriented programming continues to be one of the fundamental paradigms in the software industry. However, despite decades of its existence, a unified standard for its implementation has never been established. Modern programming languages offer a variety of object models: from the strict and static ones in Java and C# to the prototype-based in JavaScript and the minimalist in Golang. This diversity, on one hand, provides developers with freedom of choice, but on the other, creates inconveniences when applying these approaches in various situations. Most of the research conducted focuses on individual languages or syntax, without offering a systematic understanding of the architectural principles underlying their object systems.

The goal of this master's thesis is to conduct a comprehensive comparative analysis of the object models of a number of modern programming languages and, based on it, to develop an original, enhanced model. To achieve this goal, the work addresses a series of tasks: it unveils the theoretical foundations of object models, analyzes the criticism directed at OOP, and conducts a detailed analysis of the implementation of models in languages such as C#, C++, Golang,

Java, Python, Ruby, JavaScript, Scala, Smalltalk, and Zonnon, identifying their advantages, disadvantages, and typical application areas. The practical section illustrates characteristic usage errors with specific examples. The final part presents the result of a comparative analysis with the identified common trends and fundamental differences, which serves as the basis for the proposal of the author's unified object model designed to mitigate the shortcomings of existing approaches identified during the study.

The object of the study is the object models themselves, and the subject is their architectural principles, similarities, differences, and practical aspects of implementation. The methodological foundation of the work consists of theoretical methods of analysis and systematization, as well as practical methods of comparative analysis based on the study of documentation and code writing. The theoretical significance of the work lies in the systematization of knowledge about object models, while the practical significance lies in the proposal of a new, more effective model for future developments. The structure of the work includes....

Chapter 2

Literature Review

2.1 The Concept and Components of the Object Model

The object model represents a fundamental framework that defines how a programming language represents and supports objects, classes, inheritance, encapsulation, polymorphism, and other related abstractions. According to commonly accepted terminology, the term object model has two interrelated meanings: (1) the properties of objects within a particular programming language, technology, notation, or methodology that employs these objects; and (2) a set of interfaces or classes through which a program can explore and manipulate specific aspects of its environment. Examples of object models include the Java Object Model, the Component Object Model (COM), and the Object Modeling Technique (OMT). Such object models are typically defined using concepts such as class, generic function, message, inheritance, polymorphism, and encapsulation.

Cardelli and Wegner in their work “On Understanding Types, Data Abstraction, and Polymorphism,” defined object-oriented languages through three key requirements [1]. A language is considered object-oriented if it satisfies the following conditions:

- Support objects that represent data abstractions with an interface composed of named operations and an encapsulated internal state
- Objects in the language are associated with a specific object type
- Types may inherit attributes from their supertypes

These requirements were formulated in the form of a formula:

$$\textit{object - oriented} = \textit{dataabstractions} + \textit{objecttypes} + \textit{typeinheritance}$$

Booch, in his seminal work *Object-Oriented Analysis and Design with Applications*, describes the object model as a conceptual representation of the organized complexity of software systems [2].

Booch (p. 40-41) concludes that the conceptual framework for anything object-oriented is the object model—a conceptual representation of the organized complexity of software. It consists of four major elements, i.e. abstraction, encapsulation, modularity, and hierarchy. In addition, the object model contains three minor elements, i.e. typing, concurrency, and persistence.

This distinction between major and minor elements is fundamental: without the major elements, the model ceases to be object-oriented, whereas the minor elements are useful but not essential for supporting object orientation.

2.1.1 Major Components of the Object Model

The four major components of the object model form the fundamental basis on which object-oriented programming is built. Without these four elements, a programming language cannot achieve true object orientation, since they collectively determine how systems are broken down into components, organized, and managed at the conceptual level.

Abstraction

Abstraction is the process of extracting essential characteristics of an object or concept while suppressing unnecessary implementation details. As Liskov

[3] emphasizes, data abstraction is a valuable method for organizing programs to make them easier to modify and maintain.

According to Liskov's foundational work [3], a data abstraction is characterized by the separation of what an abstraction is from how it is implemented, such that implementations of the same abstraction can be substituted freely. The implementation of a data object is concerned with how that object is represented in memory, and this information is hidden from programs that use the abstraction by restricting those programs so that they cannot manipulate the representation directly, but instead must call the operations defined by the abstraction.

Encapsulation

Encapsulation is the bundling of data (attributes) and the methods that operate on that data into a single cohesive unit, typically implemented as a class. More importantly, encapsulation involves the enforcement of access controls that restrict direct manipulation of an object's internal state. This concept originated from Parnas's pioneering work on information hiding [4], which established the principle that modules should be designed such that they hide information from other modules—information that is likely to change. Parnas [4] articulated this principle by stating:

Every module in the second decomposition is characterized by its knowledge of a design decision which it hides from all others. Its interface or definition was chosen to reveal as little as possible about its inner workings.

Encapsulation hides the internal representation of an object from external access, enabling interaction with the object exclusively through its public inter-

face—a set of methods specifically provided for this purpose. Besides providing data protection, encapsulation provides a stable interface that allows developers to refactor and optimize the internal implementation without affecting dependent code.

Modularity

The principle of modularity is designing software systems as sets of discrete, independent units or components that can be designed, implemented, and maintained separately. Parnas' [4] fundamental idea of modular decomposition: modules should be designed and implemented independently, be simple enough to be fully understandable, and allow changing the implementation of a module without affecting the behavior of other modules.

In object-oriented programming, modularity comes about by packaging classes and objects into coherent units with well-defined boundaries and minimal interdependencies. According to Liskov [3], the data abstractions offer a mechanism to structure the programs into modules in which each module is responsible for implementing an abstraction.

Hierarchy

Hierarchy is the organization of abstractions into ordered structures in which more general or abstract concepts are connected to more specific or concrete concepts through “is-a” and “part of” relationships. In object-oriented programming, hierarchy is implemented using two main mechanisms: inheritance hierarchies and composition hierarchies.

In inheritance hierarchies, classes are organized into tree-like or lattice struc-

tures in which subclasses inherit attributes and behavior from their superclasses. The principle underlying such structures is Liskov's substitution principle, formulated by Liskov [3] and formalized in subsequent works [5]. This principle states that objects of a base type must be replaceable with objects of derived types without changing the correctness of the program.

In composition hierarchies, objects combine simpler objects through a “part-of” relationship, creating structures where complex objects are aggregates from simpler components. This organizational principle allows systems of any complexity to be built from well-understood, reusable building blocks. As noted in the seminal work on design patterns [6], composition provides a more flexible alternative to inheritance, allowing changes to be made to the relationships between objects at runtime that would otherwise be fixed at compile time.

Hierarchy provides several critical benefits: it enables the management of complexity through layered abstraction; it promotes code reuse through inheritance of common behavior; it establishes natural relationships between concepts that mirror real-world domain structures; and it facilitates polymorphism and dynamic dispatch, allowing for flexible, extensible program architectures.

2.1.2 Minor Components of the Object Model

The minor components are features that improve the object model's expressiveness and usability, but aren't strictly needed for a language to be considered as object-oriented. Presence of this components in modern programming languages shows the evolution of object-oriented principles for solving practical problems in modern software development.

Typing

Typing is defined a system of rules and mechanisms that determine how data types are associated with variables, expressions, and values, as well as how operations on values of different types are permitted. The type system assigns specific types to each element in the program, defining both the kind of data that can be stored and the operations that are allowed on that data.

Concurrency

Concurrency is defined as the ability of a system to manipulate several computations that are logically parallel, either through true simultaneous execution on multiprocessor systems or through alternating execution on single-processor systems. In the context of the object model, concurrency resolves the issue on how objects maintain consistency and correctness when multiple threads or processes simultaneously access and modify them.

Different object-oriented languages solve the problem of synchronizing values during parallel execution using different mechanisms. Some use mutual exclusion locks to ensure that only one thread can execute a particular method on an object at a time. Others support active objects that manage their own internal threads; still others provide transactional semantics, in which parallel operations are executed sequentially, ensuring consistency.

Persistence

Persistence refers to the ability to store and retrieve objects or their state between program runs. Without persistence, all objects created during program execution are lost when the program terminates, which limits the applicability of

object-oriented programming to stateless or short-lived computations.

The object model includes persistence through various approaches. The orthogonal persistence approach automatically saves and restores objects using special mechanisms without the need for explicit serialization code. With explicit programming, objects must implement serialization interfaces [7], or developers must write code to convert objects to storage representations and back.

2.1.3 The Selected Definition of the Object Model

Based on the conducted literature review, the following definition of the object model is adopted for the purposes of this work:

The object model is a conceptual framework that integrates a set of interrelated components — four major ones (abstraction, encapsulation, modularity, and hierarchy) and three minor ones (typing, concurrency, and persistence). It defines the means by which a programming language represents, organizes, and supports objects, classes, their interactions, inheritance relationships, mechanisms of polymorphism, and other related abstractions for the effective management of software system complexity.

2.2 Critical Analysis of Object-Oriented Programming Approach

The object-oriented programming paradigm, despite its significant influence on software development practices, demonstrates fundamental limitations and unresolved problems that hinder its applicability and effectiveness in various problem-solving areas. This section presents a systematic and critical examination of the main shortcomings of OOP. The critical perspective developed here provides a theoretical basis for evaluating the actual implementation of object models in modern programming languages.

2.2.1 Theoretical Inconsistencies in Inheritance and Subtyping

A fundamental issue in typed object-oriented languages concerns the conflation of inheritance with subtyping relations. Cook, Hill, and Canning demonstrate that inheritance, when used as a mechanism for incremental modification of recursive object definitions, does not necessarily produce subtypes [8]. The traditional assumption that inheritance hierarchies determine conformance relations proves inadequate when dealing with recursive types and contravariant method types.

The distinction between inheritance and subtyping stems from type-theoretic considerations. Cardelli and Wegner in their analysis of type systems establishes that inheritance as an implementation mechanism and subtyping represent orthogonal concerns [1]. Their concepts demonstrate that subtyping relations require adherence to the Liskov substitution principle, whereas inheritance mechanisms impose additional constraints that compromise expressiveness [9]. The consequence of this theoretical inconsistency is that inheritance-based type hierarchies

cannot enforce type safety without either compromising expressiveness or introducing type insecurities. Eiffel exemplifies this problem: by identifying classes with types and inheritance with subtyping, the language exhibits type insecurities as documented by Cardelli and Wegner [1].

The following C# example 2.1 demonstrates how inheritance does not automatically guarantee proper subtyping behavior 2.2, particularly with covariance and contravariance in method signatures:

2.2.2 The Fragile Base Class Problem

One of the most significant architectural deficiencies in inheritance-based OOP systems is the fragile base class problem, formally studied by Mikhajlov and Sekerinski [10]. This problem occurs in open object-oriented systems employing code inheritance as an implementation reuse mechanism: seemingly safe modifications to a base class can cause derived classes to malfunction, despite no explicit violation of method contracts or public interfaces. Mikhajlov and Sekerinski distinguish between two manifestations: the syntactic fragile base class problem, which necessitates recompilation of extension and client classes when base classes change, and the semantic fragile base class problem, wherein self-recursion vulnerabilities create situations where base class revisions break extension class functionality without changing the external interface [10].

The fundamental cause lies in the interaction between dynamic dispatch, self-reference (encoded as `self` or `this`), and encapsulation boundaries. When a subclass overrides methods that the base class uses internally through dynamic dispatch, changes to the base class implementation can alter method invocation sequences, causing subclass assumptions about execution order to be violated.

```
public abstract class Animal
{
    public virtual void Reproduce(Animal mate)
    {
        Console.WriteLine("Animals reproducing");
    }

    public abstract void Feed(Food food);
}

public class Dog : Animal
{
    // VIOLATION: Contravariant parameter - accepts broader type
    // This breaks Liskov Substitution Principle when used polymorphically
    public override void Reproduce(Animal mate)
    {
        if (!(mate is Dog))
            throw new ArgumentException("Dogs can only reproduce with dogs");
        Console.WriteLine("Dogs reproducing");
    }

    // VIOLATION: Covariant return would be OK, but Food parameter is problematic
    public override void Feed(Food food)
    {
        Console.WriteLine($"Dog eating: {food.Name}");
    }
}

public class Cat : Animal
{
    public override void Reproduce(Animal mate)
    {
        if (!(mate is Cat))
            throw new ArgumentException("Cats can only reproduce with cats");
        Console.WriteLine("Cats reproducing");
    }

    public override void Feed(Food food)
    {
        Console.WriteLine($"Cat eating: {food.Name}");
    }
}

public class Food { public string Name { get; set; } }
```

Fig. 2.1. Inheritance without proper subtyping

```
Animal dog = new Dog();
Animal cat = new Cat();

// This code violates the contract established by the base class
// The compiler allows it, but runtime fails - TYPE INSECURITY
try
{
    dog.Reproduce(cat); // Runtime exception: "Dogs can only reproduce with dogs"
}
catch (ArgumentException ex)
{
    Console.WriteLine($"Type safety violation: {ex.Message}");
}
```

Fig. 2.2. Inheritance without proper subtyping: manifestation

This coupling violation fundamentally undermines the promise of modular reuse through inheritance. In open systems where subclasses are developed independently of base class implementations, predicting the consequences of base class modifications becomes impractical [10].

2.2.3 Excessive Coupling and Tight Interdependencies

The inheritance mechanism introduces structural coupling between base and derived classes that compromises modularity and reusability. Booch's analysis of class quality metrics identifies coupling as a critical measure of design quality, noting that inheritance creates strong coupling between superclasses and subclasses [2]. This coupling creates dependencies where modifications in parent classes propagate through inheritance chains, a phenomenon that Hitz and Montazeri characterize as particularly problematic: excessive coupling between object classes is detrimental to modular design and prevents reuse of individual components in alternative applications [11].

Beyond inheritance-induced coupling, OOP systems exhibit architectural issues with circular dependencies among classes. Circular dependencies create

```
public class BankAccount
{
    protected decimal _balance;

    public virtual void Deposit(decimal amount)
    {
        _balance += amount;
        Console.WriteLine($"Deposited {amount}. Balance: {_balance}");
    }

    public virtual void Withdraw(decimal amount)
    {
        if (amount <= _balance)
        {
            _balance -= amount;
            Console.WriteLine($"Withdrawn {amount}. Balance: {_balance}");
        }
    }

    public virtual decimal GetBalance()
    {
        return _balance;
    }
}
```

Fig. 2.3. Fragile Base Class: first implementation base class

tight mutual coupling that reduces the possibility of separate reuse and creates domino effects where changes in one module spread unwanted side effects to dependent modules. This architectural pattern violates the foundational principle of modular decomposition, particularly problematic in large-scale systems where dependency management becomes increasingly complex.

The Gang of Four Design Patterns documentation explicitly acknowledges this limitation, stating that inheritance exposes a subclass to details of its parent's implementation, and therefore "inheritance breaks encapsulation" [6]. The authors warn that implementation of a subclass can become so bound to the implementation of its parent that any change in the parent's implementation forces the subclass to change. They further note that polymorphism combined with implementation inheritance can create systems where it is difficult to predict which

```
public class PremiumAccount : BankAccount
{
    private decimal _monthlyFee = 10m;
    private int _freeTransactions = 10;
    private int _transactionCount = 0;

    public override void Deposit(decimal amount)
    {
        _transactionCount++;
        if (_transactionCount > _freeTransactions)
        {
            base.Deposit(amount - _monthlyFee); // Account for fee
        }
        else
        {
            base.Deposit(amount);
        }
    }

    public override void Withdraw(decimal amount)
    {
        _transactionCount++;
        base.Withdraw(amount); // Assumes Withdraw will update _balance
    }

    public void ApplyMonthlyFee()
    {
        _balance -= _monthlyFee;
    }
}
```

Fig. 2.4. Fragile Base Class: first implementation deriving class

```
public class BankAccount_V2
{
    protected decimal _balance;
    protected decimal _totalFees = 0;

    // CHANGE: Added automatic fee application
    public virtual void Deposit(decimal amount)
    {
        _balance += amount;
        ApplyTransactionFee(); // NEW: automatic fee
        Console.WriteLine($"Deposited {amount}. Balance: {_balance}");
    }

    public virtual void Withdraw(decimal amount)
    {
        if (amount <= _balance)
        {
            _balance -= amount;
            ApplyTransactionFee(); // NEW: automatic fee
            Console.WriteLine($"Withdrawn {amount}. Balance: {_balance}");
        }
    }

    protected virtual void ApplyTransactionFee()
    {
        _balance -= 1m; // Transaction fee
        _totalFees += 1m;
    }

    public virtual decimal GetBalance()
    {
        return _balance;
    }
}
```

Fig. 2.5. Fragile Base Class: misspelled implementation base class


```

public class PremiumAccount_V2 : BankAccount_V2
{
    private decimal _monthlyFee = 10m;
    private int _freeTransactions = 10;
    private int _transactionCount = 0;

    public override void Deposit(decimal amount)
    {
        _transactionCount++;
        if (_transactionCount > _freeTransactions)
        {
            // PROBLEM: Base.Deposit now also applies transaction fee!
            // Double deduction: _monthlyFee + automatic ApplyTransactionFee()
            base.Deposit(amount - _monthlyFee);
        }
        else
        {
            base.Deposit(amount); // Unexpected fee added by base class
        }
    }

    public override void Withdraw(decimal amount)
    {
        _transactionCount++;
        base.Withdraw(amount); // Unexpected fee added by base class
    }
}

```

Fig. 2.6. Fragile Base Class: misspelled implementation deriving class

```

var account = new PremiumAccount_V2();
account.Deposit(100); // Expected: balance = 100; Actual: balance = 89 (100 - 10 - 1)
Console.WriteLine($"Balance: {account.GetBalance()}"); // Shows unexpected deduction

// Subclass assumptions about execution order are violated
// The fragile base class problem manifests as silent data corruption

```

Fig. 2.7. Fragile Base Class: demonstration

method will be invoked in response to a message, complicating both debugging and maintenance. architectural coupling

```
public class Account {  
    protected double balance; // Exposed implementation detail  
  
    public Account(double initialBalance) {  
        this.balance = initialBalance;  
    }  
  
    public void deposit(double amount) {  
        if (amount <= 0) {  
            throw new IllegalArgumentException("Amount must be positive");  
        }  
        balance += amount;  
        logTransaction("DEPOSIT", amount);  
    }  
  
    protected void logTransaction(String type, double amount) {  
        System.out.printf("[%s] %.2f | Balance: %.2f%n",  
            type, amount, balance);  
    }  
}
```

Fig. 2.8. Base class Account exposing internal state and workflow through protected members

The examples demonstrate two critical manifestations of excessive coupling in object-oriented systems. Figures 2.8 and 2.9 illustrate inheritance-induced coupling where SavingsAccount becomes dependent on implementation details of its parent Account: the protected field balance, the internal workflow sequence (deposit → log), and the assumption that logTransaction() executes after state modification. This breaks encapsulation because the subclass gains access to superclass internals that should remain hidden. Critically, any modification to Account's implementation—renaming balance, reordering workflow steps, or changing logTransaction()'s signature—necessitates corresponding changes in SavingsAccount, creating ripple effects through the inheritance hierarchy. Figures 2.10 2.11 2.12 exhibit architectural coupling through circular dependencies

```

public class SavingsAccount extends Account {
    private static final double INTEREST_RATE = 0.05;

    public SavingsAccount(double initialBalance) {
        super(initialBalance);
    }

    @Override
    public void deposit(double amount) {
        super.deposit(amount);
        applyInterest(); // Breaks if parent changes workflow order
    }

    private void applyInterest() {
        // Direct manipulation of protected state violates encapsulation
        double interest = balance * INTEREST_RATE;
        balance += interest;
        logTransaction("INTEREST", interest);
    }
}

```

Fig. 2.9. Derived class SavingsAccount tightly coupled to parent's implementation details

```

public class Order {
    private Customer customer;
    private List<OrderItem> items;

    public Order(Customer customer) {
        this.customer = Objects.requireNonNull(customer);
        this.items = new ArrayList<>();
        // Creates circular dependency: Order  $\rightarrow$  Customer
        customer.registerOrder(this); // Customer  $\rightarrow$  Order
    }

    public void addItem(OrderItem item) {
        items.add(item);
    }

    public double calculateTotal() {
        return items.stream()
            .mapToDouble(OrderItem::getPrice)
            .sum();
    }

    public Customer getCustomer() {
        return customer;
    }
}

```

Fig. 2.10. Class Order establishing bidirectional dependency with Customer

```
public class Customer {
    private String id;
    private List<Order> orders;

    public Customer(String id) {
        this.id = id;
        this.orders = new ArrayList<>();
    }

    public void registerOrder(Order order) {
        orders.add(order);
    }

    public double getLifetimeValue() {
        return orders.stream()
            .mapToDouble(Order::calculateTotal)
            .sum();
    }
}
```

Fig. 2.11. Class Customer completing circular dependency cycle and depending on Order internals

```
public class OrderItem {
    private String productId;
    private double price;
    private int quantity;

    public OrderItem(String productId, double price, int quantity) {
        this.productId = productId;
        this.price = price;
        this.quantity = quantity;
    }

    public double getPrice() {
        return price * quantity; // Implementation detail exposed to callers
    }
}
```

Fig. 2.12. Supporting class OrderItem whose internal calculation logic is exposed to dependent classes

between `Order` and `Customer`, where each class maintains a direct reference to the other. This mutual dependency prevents independent reuse of either component, violates modular decomposition principles, and creates domino effects where changes to `Order`'s calculation logic (`calculateTotal()`) propagate unpredictably to `Customer.getLifetimeValue()`.

2.2.4 Multiple Inheritance and Ambiguity Resolution

Languages supporting multiple inheritance encounter the well-documented diamond problem, wherein a derived class inheriting from multiple parents creates ambiguity about which implementation of inherited methods should be invoked when those parents themselves share a common ancestor. This problem reflects fundamental challenges in the inheritance model: the attempt to linearize and resolve multiple method resolution paths introduces complexity that defeats the transparency and simplicity promised by object-oriented design.

Languages addressing this through single inheritance restrictions or interface-based approaches acknowledge that inheritance as a reuse mechanism creates structural problems that require linguistic constraints. The very fact that languages must prohibit or heavily restrict inheritance patterns suggests limitations in the fundamental mechanism itself.

In the C++ implementation (Figs. 2.13, 2.14, 2.15, 2.16), class `Bat` contains two distinct subobjects of `Animal`—one through the `Mammal` path and another through the `Bird` path—each maintaining independent state (`health`). This duplication violates the object identity principle: a bat logically possesses a single `health` value, yet the inheritance model creates two physically separate instances. Access to inherited members becomes ambiguous at compile time, requiring ex-

```
class Animal {  
protected:  
    int health; // Shared state from common ancestor  
public:  
    Animal() : health(100) {}  
    void breathe() {  
        std::cout << "Breathing with health=" << health << std::endl;  
    }  
};
```

Fig. 2.13. Base class Animal defining shared state and behavior in diamond inheritance hierarchy

```
class Mammal : public Animal {  
    // NO override of breathe(); inherits Animal::breathe() directly  
    // NO modification of health initialization  
};
```

Fig. 2.14. Intermediate class Mammal inheriting from Animal without overriding base methods

```
class Bird : public Animal {  
    // NO override of breathe(); inherits Animal::breathe() directly  
    // NO modification of health initialization  
};
```

Fig. 2.15. Intermediate class Bird inheriting from Animal without overriding base methods

```
class Bat : public Mammal, public Bird {  
};  
  
int main() {  
    Bat bat;  
    bat.breathe(); // COMPILATION ERROR: ambiguous call  
    bat.health = 50; // COMPILATION ERROR: ambiguous member access  
}
```

Fig. 2.16. Derived class Bat exhibiting diamond inheritance ambiguity without virtual inheritance

```
class Animal:
    def __init__(self):
        self.health = 100 # State defined in common ancestor

    def breathe(self):
        return f"Breathing (health={self.health})"
```

Fig. 2.17. Base class Animal defining shared state in Python diamond hierarchy

```
class Mammal(Animal):
    def __init__(self):
        super().__init__()
        # NO override of breathe(); delegates to Animal.breathe()
```

Fig. 2.18. Intermediate class Mammal inheriting from Animal without method override

```
class Bird(Animal):
    def __init__(self):
        super().__init__()
        # NO override of breathe(); delegates to Animal.breathe()
```

Fig. 2.19. Intermediate class Bird inheriting from Animal without method override

```
class Bat(Mammal, Bird):
    def __init__(self):
        super().__init__()

bat = Bat()
print(bat.breathe())
print(Bat.__mro__) # Shows linearization path
# (<class '__main__.Bat'>, <class '__main__.Mammal'>,
# <class '__main__.Bird'>, <class '__main__.Animal'>, <class 'object'>)
```

Fig. 2.20. Derived class Bat demonstrating method resolution order limitations in Python

explicit path qualification (`bat.Mammal::breathe()`) that exposes implementation details and defeats encapsulation. The Python implementation (Figs. 2.17, 2.18, 2.19, 2.20) mitigates lookup ambiguity through C3 linearization, but the fundamental conceptual problem persists: multiple inheritance paths to a single logical ancestor create tension between the inheritance graph's topology and the linear method resolution order.

2.2.5 The Reusability Paradox

Despite promises of enhanced reusability through inheritance and polymorphism, empirical software development experience reveals significant obstacles to achieving code reuse in OOP systems. Graham observes that object-oriented programming offers “a sustainable way to write spaghetti code,” suggesting that the anticipated reusability gains often fail to materialize [12]. The problem lies partly in the tightly coupled nature of inheritance-based designs: inheriting from an existing class binds the derived class not just to the inherited interface but to the entire implementation dependency graph of the parent class.

Effective reuse typically requires either shallow inheritance hierarchies or careful extraction of orthogonal concerns into separately reusable components. This requirement contradicts the hierarchical organization that inheritance encourages, creating a tension between the inheritance mechanism and practical reusability goals.

The inheritance-based approach in Fig. 2.21 exemplifies the reusability paradox: despite leveraging inheritance to extend `HashSet`'s functionality, the derived class becomes tightly coupled to implementation details invisible in the public interface. Specifically, `HashSet.addAll()` internally invokes `add()` for each


```
1 // Problematic reuse through inheritance
2 public class InstrumentedHashSet<E> extends HashSet<E> {
3     private int addCount = 0;
4
5     @Override
6     public boolean add(E e) {
7         addCount++;
8         return super.add(e);
9     }
10
11     @Override
12     public boolean addAll(Collection<? extends E> c) {
13         addCount += c.size();
14         return super.addAll(c);
15     }
16
17     public int getAddCount() {
18         return addCount;
19     }
20 }
```

Fig. 2.21. Instrumented set implementation using inheritance

element—a behavioral detail not specified in the `Set` interface contract. Consequently, when clients invoke `addAll({'apple', 'banana', 'cherry'})`, the counter increments six times (three via the overridden `addAll()` and three via subsequent `add()` calls), violating expectations. The composition-based solution in Fig. 2.22 achieves reliable reuse by decoupling instrumentation logic from the underlying set implementation through explicit delegation.

2.2.6 State Management Complexity and Object Identity

Object-oriented systems organize code around mutable state encapsulated within objects. This approach fundamentally differs from functional programming's emphasis on immutability and pure functions. The combination of mutable state, object identity, and encapsulation creates complexity in reasoning about program behavior. When objects maintain mutable internal state, analyzing program correctness requires understanding not only the method calls but

```
1 // Robust reuse through composition
2 public class InstrumentedSet<E> implements Set<E> {
3     private final Set<E> delegate;
4     private int addCount = 0;
5
6     public InstrumentedSet(Set<E> delegate) {
7         this.delegate = Objects.requireNonNull(delegate);
8     }
9
10    @Override
11    public boolean add(E e) {
12        addCount++;
13        return delegate.add(e);
14    }
15
16    @Override
17    public boolean addAll(Collection<? extends E> c) {
18        addCount += c.size();
19        return delegate.addAll(c);
20    }
21
22    public int getAddCount() {
23        return addCount;
24    }
25
26    // Delegation to all other Set methods omitted for brevity
27    @Override public int size() { return delegate.size(); }
28    @Override public boolean isEmpty() { return delegate.isEmpty(); }
29    // ... remaining Set interface methods delegated to 'delegate'
30 }
```

Fig. 2.22. Instrumented set implementation using composition

also the complete history of state modifications, increasing the difficulty of formal verification and testing.

Reynolds' distinction between values (immutable, state-based equality) and entities (distinct identity, mutable state) illustrates the conceptual tension in OOP [13]. In value-oriented systems, equality testing is unambiguous and immutability simplifies reasoning. In object-oriented systems, the notion of object identity introduces questions about whether two references point to the same object or equivalent objects, complications that functional approaches avoid entirely through immutable values.

Furthermore, the emphasis on object identity creates problems in concurrent systems where mutable shared state becomes a liability. Multiple threads attempting to access and modify object state require extensive synchronization, complicating concurrent programming compared to functional approaches based on immutable data and pure transformations [13].

Figure 2.23 defines `BankAccount` as an entity with mutable state (`balance`) and dual equality semantics: reference identity (`==`) and state-based equivalence (`.equals()`). Figure 2.24 demonstrates the resulting reasoning complexity. Lines 5–14 show how shared identity causes non-local side effects: mutation through reference `a` implicitly alters the state observed through `b`, requiring the programmer to track aliasing relationships. Lines 16–27 reveal following problem: initially equivalent entities `c` and `d` diverge after mutation, that is violating referential transparency.

```
public class BankAccount {
    private final String accountId;
    private int balance;

    public BankAccount(String accountId, int initialBalance) {
        this.accountId = accountId;
        this.balance = initialBalance;
    }

    public void deposit(int amount) {
        this.balance += amount;
    }

    public void withdraw(int amount) {
        this.balance -= amount;
    }

    public int getBalance() {
        return this.balance;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof BankAccount)) return false;
        BankAccount other = (BankAccount) obj;
        return this.accountId.equals(other.accountId) &&
            this.balance == other.balance;
    }
}
```

Fig. 2.23. Mutable entity class with identity and state-based equality semantics

```
public class AccountManager {
    public static void demonstrateIdentityProblem() {
        // Two references to the SAME entity (identity equivalence)
        BankAccount a = new BankAccount("A123", 100);
        BankAccount b = a;

        assert (a == b);           // true: same identity
        assert a.equals(b);        // true: same state

        a.deposit(50);

        // Mutation affects BOTH references due to shared identity
        assert a.getBalance() == 150;
        assert b.getBalance() == 150; // Unexpected side effect!

        // Two DISTINCT entities with identical initial state (value equivalence)
        BankAccount c = new BankAccount("B456", 100);
        BankAccount d = new BankAccount("B456", 100);

        assert (c != d);           // false: different identities
        assert c.equals(d);        // true: equivalent state

        c.deposit(50);

        // Mutation breaks state equivalence despite identical initial state
        assert !c.equals(d);       // State divergence after mutation
        assert c.getBalance() == 150;
        assert d.getBalance() == 100; // Independent state evolution
    }
}
```

Fig. 2.24. Demonstration of identity-value tension and state mutation effects

2.2.7 Absence of Clear Separation Between Specification and Implementation

OOP languages typically conflate class specifications with implementation details through a single construct. Unlike languages supporting explicit separation between interfaces and implementations, OOP classes bundle these concerns, making it difficult to understand what behavior clients should depend upon versus what represents implementation-specific detail. This conflation particularly affects evolution and maintenance: modifications to implementation details that preserve external contracts can nonetheless break derived classes through the fragile base class problem.

2.2.8 Empirical Evidence of Design Difficulties

The widespread adoption of design patterns and anti-pattern literature documents systematic difficulties with OOP system design. The Gang of Four patterns book identifies recurring design problems that OOP practitioners encounter; the very necessity of these patterns suggests that raw OOP mechanisms inadequately support common design requirements [14]. Anti-patterns such as the God Object problem reflect systematic tendency toward violation of single responsibility principles.

Chidamber and Kemerer's metrics suite for measuring OOP design quality reveals correlations between high metric values and undesirable properties: high coupling between object classes indicates fault-proneness and maintenance difficulty; excessive inheritance tree depth indicates reduced comprehensibility; high weighted methods per class indicates complexity [15]. These metrics-based findings empirically validate concerns about OOP's inherent tendency toward

```
abstract class AbstractEnterpriseProcessor {  
  
    protected AuditService audit;  
    protected PricingService pricing;  
    protected ValidationService validation;  
  
    public final Order process(OrderRequest request) {  
  
        validate(request);  
        preProcess(request);  
  
        double price = pricing.calculate(request);  
        Order order = createOrder(request, price);  
  
        postProcess(order);  
        audit.log("order_created");  
  
        return order;  
    }  
  
    protected void validate(OrderRequest request) {  
        validation.validateUser(request.userId());  
        validation.validateItems(request.items());  
    }  
  
    protected abstract void preProcess(OrderRequest request);  
    protected abstract Order createOrder(OrderRequest request, double price);  
    protected abstract void postProcess(Order order);  
}
```

Fig. 2.25. Base class implementing the Template Method pattern for order processing.

coupled, complex designs. Their research demonstrates that systems exhibiting high CBO values require more rigorous testing and experience higher defect rates, a pattern consistent across multiple industrial software projects [15].

The example presented in Figures 2.25,2.26,2.27,2.28,2.29 illustrates structural characteristics empirically associated with reduced maintainability and increased defect proneness in object-oriented systems.

First, the hierarchy exhibits a Depth of Inheritance Tree (DIT) equal to five. Understanding the runtime behavior of the `process` operation requires analysis across multiple abstraction layers. It has been empirically correlated with de-

```
abstract class PaymentProcessor extends AbstractEnterpriseProcessor {  
  
    protected PaymentGateway payment;  
    protected TransactionRepository transactions;  
  
    @Override  
    protected Order createOrder(OrderRequest request, double price) {  
  
        payment.charge(request.userId(), price);  
        transactions.save(request.userId(), price);  
  
        return new Order(request.userId(), price);  
    }  
}
```

Fig. 2.26. Inheritance layer introducing payment and transaction persistence behavior.

created comprehensibility and increased maintenance effort.

Second, the final concrete class demonstrates high Coupling Between Object classes (CBO). The EnterpriseOrderProcessor directly depends on numerous infrastructural and domain services. When combined with inherited dependencies from ancestor classes, the total number of external collaborators exceeds fifteen distinct services. High CBO values have been consistently associated with increased fault proneness and testing complexity.

Third, the concrete processor exhibits characteristics of the God Object anti-pattern. It assumes responsibility for heterogeneous business concerns. This concentration of responsibilities violates the Single Responsibility Principle and increases the likelihood of ripple effects during modification.

2.2.9 Systematic Classification of OOP Limitations

The critical examination conducted in the preceding subsections reveals a coherent set of fundamental limitations inherent in the object-oriented programming paradigm. These limitations can be systematically classified into several


```
abstract class FraudAwareProcessor extends PaymentProcessor {  
  
    protected FraudService fraud;  
    protected BlacklistService blacklist;  
  
    @Override  
    protected void preProcess(OrderRequest request) {  
  
        double score = fraud.score(request);  
  
        if (score > 0.8) {  
            blacklist.add(request.userId());  
            throw new RuntimeException("fraud suspected");  
        }  
    }  
}
```

Fig. 2.27. Inheritance layer adding fraud detection and blacklist management.

```
abstract class RetryableProcessor extends FraudAwareProcessor {  
  
    protected RetryService retry;  
  
    @Override  
    public final Order process(OrderRequest request) {  
  
        return retry.execute(() -> super.process(request));  
    }  
}
```

Fig. 2.28. Inheritance layer introducing retry semantics around the processing algorithm.

interconnected categories, which collectively explain the challenges encountered in the design, implementation, and maintenance of large-scale OOP systems.

1. **Theoretical and Type-System Deficiencies:** At the most foundational level, OOP suffers from a conflation of distinct concepts, primarily inheritance (an implementation reuse mechanism) and subtyping (a type compatibility relation). This conflation, formalized by type theory, leads to inherent tensions between expressiveness and type safety, as exemplified by the need for complex workarounds.

2. **Architectural and Structural Deficiencies:** The inheritance mechanism itself introduces severe architectural flaws. It creates tight, often hidden, coupling between base and derived classes, fundamentally breaking encapsulation. This results in the well-documented *Fragile Base Class Problem*, where the modularity of a system is compromised because changes in one module (a base class) can unpredictably break functionality in dependent modules (derived classes), despite adherence to public interfaces.
3. **Compositional and Reusability Deficiencies:** Contrary to its core promises, OOP often hinders effective code reuse. The *Reusability Paradox* highlights that deep inheritance hierarchies bind classes to extensive implementation dependency graphs, making isolated reuse difficult. Furthermore, the challenges of *Multiple Inheritance*, such as the diamond problem, demonstrate the paradigm's struggle to manage complexity when composing behaviors from multiple sources.
4. **State Management and Concurrency Deficiencies:** The paradigm's emphasis on mutable state and object identity complicates reasoning about program behavior, especially in concurrent and distributed environments. The need to manage and synchronize shared mutable state stands in stark contrast to the simplicity offered by immutable data models, making OOP a suboptimal choice for highly concurrent systems.
5. **Design and Evolvability Deficiencies:** The absence of a clear separation between specification and implementation within the class construct impedes system evolution and maintenance. This, combined with the inherent coupling, leads to well-documented design anti-patterns. Empirical evi-

dence, including software metrics and the proliferation of design patterns, confirms a systematic tendency toward complex, tightly coupled designs that are fault-prone and difficult to comprehend.

```
class EnterpriseOrderProcessor extends RetryableProcessor {  
  
    private InventoryService inventory;  
    private RecommendationEngine recommendations;  
    private ERPClient erp;  
    private NotificationService notifications;  
    private AnalyticsService analytics;  
    private DiscountService discounts;  
    private LoyaltyService loyalty;  
    private ShippingService shipping;  
    private InvoiceService invoice;  
    private CRMClient crm;  
  
    @Override  
    protected void postProcess(Order order) {  
  
        inventory.reserve(order);  
        shipping.schedule(order);  
        invoice.generate(order);  
        erp.sync(order);  
        crm.update(order);  
  
        notifications.send(order);  
        recommendations.update(order);  
        analytics.track(order);  
        loyalty.addPoints(order);  
  
        applyDiscountPolicies(order);  
        triggerCampaigns(order);  
        reconcileAccounting(order);  
    }  
  
    private void applyDiscountPolicies(Order order) { }  
    private void triggerCampaigns(Order order) { }  
    private void reconcileAccounting(Order order) { }  
}
```

Fig. 2.29. Concrete enterprise processor exhibiting extensive infrastructure coupling.

2.3 Review of Related Work

The problem of comparative analysis of programming languages is not new to computer science. Over recent decades, numerous works have been published dedicated to the classification and comparison of object-oriented languages. Through a systematic literature review, we have identified three main groups of studies: fundamental taxonomies, empirical performance comparisons, and narrowly focused comparative analyses of specific language pairs.

2.3.1 Fundamental taxonomies and theoretical foundations

The foundational works in the classification of object models were conducted in the late 1980s. The seminal work by P. Wegner, “Dimensions of Object-Oriented Language Design” [16], laid the groundwork by introducing strict criteria for “object-based” languages. However, contemporary research indicates that these hierarchical taxonomies are losing their relevance in the era of multi-paradigm languages.

In a recent work, M. Vandeloise [17] argues that traditional classifications fail to account for hybrid languages (e.g., Scala, Rust) and proposes a “compositional reconstruction” of paradigms based on type theory, where the key factor becomes safety guarantees rather than the presence of classes. This is supported by research from Crichton et al. [18], who propose treating the ownership semantics in Rust as a new fundamental dimension of the object model, which ensures memory safety without a garbage collector—an aspect absent from earlier taxonomies like Armstrong’s [19].

2.3.2 Empirical Performance Comparisons

A second extensive group of works is dedicated to the quantitative comparison of languages. The well-known study by L. Prechelt [20] compares seven languages (C, C++, Java, Python, among others) based on criteria such as execution time and memory consumption. Similar contemporary studies, for instance, the work by Farooq and Khan [21], propose frameworks for assessing the popularity and technical efficiency of languages.

A key limitation of this group of works, in relation to the goals of our dissertation, is their focus on **quantitative metrics** (speed, code volume) rather than on the **architectural distinctions** between object models. They answer the question “which language is faster,” but do not explain how differences in the implementation of v-tables, prototypes (JavaScript), or structural typing (Go) influence the architecture of software systems.

2.3.3 Narrowly Focused and Engineering Comparisons

The third category encompasses works that compare specific language pairs or their individual mechanisms. Numerous publications exist that contrast Go’s interface model with the class-based model of Java/C++ [22], [23]. Hybrid functional-object-oriented models in Scala compared to Java have also been extensively documented [24].

Nevertheless, a significant gap remains in the academic literature regarding works that conduct a comprehensive, end-to-end analysis of the entire spectrum of modern models: ranging from “pure“ OOP (Smalltalk, Ruby) to prototype-based (JavaScript), hybrid (Scala, C++), and active object models (Zonnon). The Zonnon language and its unique model, based on active objects and dialogs as

described by Gutknecht [25], receives virtually no attention in broad comparative surveys, remaining confined to specialized niche literature.

2.3.4 Conclusion

The analysis of the literature demonstrates that, despite the abundance of comparative studies, a comprehensive investigation is lacking—one that would:

1. Unify both classical and modern languages within a single comparative framework.
2. Examine not only syntax or performance but also the internal implementation of object models (dispatch mechanisms, memory management, object layouts).
3. Identify the architectural consequences of choosing a particular model for designing complex systems.

This dissertation aims to fill this gap by proposing a systematization that includes both industry standards and academic languages, with the goal of developing recommendations for an enhanced object model.

2.4 Methodology of Comparative Analysis

When developing object models for programming language, a systematic and reproducible approach to their analysis must be employed. This section provides a list of criteria for comparing the implementation of object models.

2.4.1 Rationale for the Chosen Methodology

The selection of comparison criteria is justified by a number of fundamental works on programming language design. Cardelli and Wegner [1] established three key requirements for defining an object-oriented language: support for objects as data abstractions, the association of objects with a specific type, and type inheritance. These requirements form a basic classification of languages and allow one to distinguish object-oriented languages from those that merely support some object-based features.

Booch [2] defined four core elements of the object model — abstraction, encapsulation, modularity, and hierarchy — as well as three secondary elements — as well as three secondary elements—typing, concurrency, and persistence. According to Booch, without the core elements, a model ceases to be object-oriented; however, the secondary elements significantly impact the model's practical applicability. This hierarchical approach to identifying elements helps determine which language characteristics are critical.

The methodology for evaluating programming languages is based on a comprehensive approach discussed in works on language assessment criteria. Sebesta [26] proposed four primary criteria for evaluating languages: readability, writability, reliability, and cost. Furthermore, readability and writability depend on a wide range of language characteristics, including syntactic simplicity, support for

abstraction, and orthogonality. Modern works on code quality and software maintainability emphasize the importance of metrics such as modularity, reusability, analyzability, and modifiability [27], which are directly influenced by the characteristics of a specific language's object model.

2.4.2 Object Model Comparison Criteria

Based on the fundamental works mentioned above and an analysis of programming language standards, the following criteria have been identified for the systematic comparison of object models:

1. **Type System:** allows to evaluate how the language ensures code reliability and builds its infrastructure through data validation rules.
2. **Inheritance Mechanism:** is necessary to compare how languages support code reuse and hierarchy organization, which directly affects maintainability and extensibility.
3. **Abstraction and Encapsulation:** helps analyze how language manages complexity by hiding implementation details and protecting the internal state of objects.
4. **Polymorphism:** comparing the trade-off between performance (static dispatching) and code flexibility (dynamic dispatching) in different languages.
5. **Object Creation Mechanism:** Compare fundamental approaches to defining objects: based on classes (instances) or prototypes (delegation).
6. **Metaclasses and Metaprogramming:** assessments of the language's ability to introspect and modify its own structure during execution.

7. **Multiple Inheritance and Mixins:** Compare alternative mechanisms for composition and extending functionality beyond a strict class hierarchy.
8. **Exception Handling:** Compare the philosophies of languages regarding error handling guarantees: strict control at the compilation stage (checked exceptions) or a more flexible runtime model.
9. **Modularity and Packages:** assess how the language supports the organization and scaling of large projects through code structuring mechanisms.
10. **Subtyping and Contract model:** Comparison of how languages relate inheritance and subtyping, and what means of contract formalization they provide (invariants, preconditions, postconditions).
11. **Composition mechanisms and alternatives to inheritance:** Comparison of the degree of composition support (delegation, decorators, first-class interfaces) as an alternative to inheritance, it shows how much the model really supports reuse without hierarchies.
12. **Managing the mutability and evolution of hierarchies:** Languages protect clients from base class changes in different ways: final/sealed constructs, closed hierarchies, explicit extension points, and redefinition rules.
13. **Multi-paradigm integration:** Comparison of how the object model is consistent with other paradigms (immutability-first, ownership, actors, active objects).
14. **Support for formal verifiability and specification:** Compare the extent to which the object model is suitable for formal methods: the presence of strict type semantics, restrictions on side effects, explicit specification interfaces.

2.4.3 Methodology of Comparative Analysis

Based on the reviewed literature and the theoretical foundations presented above, the comparative methodology in this dissertation is reduced to a set of eight core criteria. Each criterion reflects a structural dimension of the object model rather than superficial syntactic or performance characteristics.

1. Type System and Subtyping Model

The type system constitutes the primary semantic framework of an object model. Cardelli and Wegner emphasize that understanding object-oriented languages requires distinguishing between data abstraction, polymorphism, and subtyping [1]. They explicitly separate inheritance from subtyping, demonstrating that “inheritance is not subtyping” and that type safety depends on formal subtyping relations rather than reuse mechanisms.

Therefore, the comparison must evaluate: (i) nominal vs. structural typing, (ii) variance rules, and (iii) the formal relation between inheritance and subtyping.

Relevance: The type system defines semantic correctness boundaries and impacts modular reasoning.

2. Abstraction and Encapsulation Mechanisms

Abstraction and encapsulation are identified by Booch as core elements of the object model [2]. Parnas’ principle of information hiding states that modules should conceal design decisions likely to change [4].

Encapsulation mechanisms (visibility modifiers, module boundaries, interface constructs) determine whether internal representation is truly protected or merely conventionally hidden. Weak encapsulation leads to fragile hierarchies

and tight coupling, as documented in design pattern literature.

Relevance: Encapsulation quality determines modular stability and evolvability.

3. Inheritance Semantics and Hierarchy Management

Hierarchy is one of Booch's four major components, yet inheritance introduces well-documented structural risks. Mikhajlov and Sekerinski formally analyze the fragile base class problem, demonstrating that "apparently safe modifications to a base class can cause derived classes to malfunction" [10].

Consequently, the methodology evaluates: (i) single vs. multiple inheritance, (ii) linearization rules, (iii) mechanisms such as sealed/final classes, and (iv) protection against fragile base class effects.

Relevance: Inheritance directly affects modular soundness and long-term maintainability.

4. Composition and Alternatives to Inheritance

Because class inheritance exposes the implementation details of superclasses to subclasses, many programmers prefer composition. Therefore, object models must be compared with respect to delegation mechanisms, interface-based composition, mixins, and traits. This criterion evaluates whether the language provides first-class constructs enabling reuse without hierarchical fragility.

Relevance: Composition mechanisms mitigate structural deficiencies of inheritance-based reuse.

5. Object Representation and Creation Model

The languages diverge fundamentally in how objects are constructed and represented: class-based instantiation, prototype-based delegation, or hybrid models.

This dimension determines object identity semantics, dispatch strategy, and layout constraints. It also affects meta-level extensibility and runtime adaptability.

Relevance: Object construction semantics shape the entire runtime model.

6. Polymorphism and Dispatch Mechanisms

Polymorphism is central to OOP theory. The comparison must distinguish between static (compile-time) and dynamic (runtime) dispatch and evaluate trade-offs between flexibility and predictability.

Relevance: Dispatch rules determine behavioral substitutability and performance characteristics.

7. State Model, Mutability, and Concurrency Guarantees

Reynolds distinguishes between values and entities, highlighting the conceptual implications of mutable object identity [13]. Mutable shared state complicates reasoning, particularly under concurrency.

Languages differ in whether they enforce immutability by default, provide ownership models, or rely on synchronization primitives. This dimension is essential for evaluating architectural scalability and suitability for concurrent systems.

Relevance: Increasingly critical in modern multi-core environments.

8. Support for Formal Specification and Verifiability

The theoretical separation between behavioral subtyping and implementation inheritance [8] implies the need for explicit contracts. Languages that support invariants, preconditions, and postconditions allow formal reasoning about substitutability.

The absence of clear separation between specification and implementation has been identified as a structural weakness of mainstream OOP systems [10]. Thus, the methodology assesses the availability of contract models and formal type semantics.

Relevance: Essential for long-term correctness, static verification, and architectural reliability.

Excluded Criteria

Certain criteria from the initial list were excluded:

- *Exception handling* — although relevant to language philosophy, it does not directly define the object model architecture.
- *Packages and modular organization* — considered secondary to encapsulation and hierarchy, already captured under modular abstraction.
- *Persistence* — treated as an orthogonal runtime concern rather than a defining structural property.
- *Metaprogramming* — important for language flexibility but not foundational for object model semantics.

Chapter 3

Methodology

This chapter provides a comparative analysis of object models in modern programming languages. The study covers a representative group of languages — C#, C++, Golang, Java, Python, Ruby, JavaScript, Scala, Smalltalk, and Zonnon— to encompass a wide range of object-oriented paradigms, from classical, class-based, to prototyping and structural typing systems. The analytical method includes a systematic comparison of the object model of each language in accordance with an established set of criteria based on the theoretical foundation presented in the previous chapter. In addition, to substantiate the practical consequences of theoretical differences and identified potential shortcomings, the methodology includes qualitative verification of documented cases of errors. This verification includes the analysis of real software defects and anti-patterns that can be directly related to specific characteristics or limitations within the frame of the implementation of the object model of the corresponding language

3.1 Comparison Technique

3.1.1 Type System and Subtyping Model

Metric Definition

Let the *Type System and Subtyping Soundness Index (TSSI)* be defined as:

$$TSSI(L) = \alpha \cdot S_{nom}(L) + \beta \cdot S_{var}(L) + \gamma \cdot S_{rel}(L) \quad (3.1)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Nominal–Structural Typing Score $S_{nom}(L)$

$$S_{nom}(L) = \frac{I_{struct}(L) + I_{nom}(L)}{2} \quad (3.2)$$

where:

- $I_{struct}(L) = 1$ if the language supports structural subtyping with formal typing rules; 0 otherwise.
- $I_{nom}(L) = 1$ if the language supports nominal typing with explicit subtype declarations; 0 otherwise.

Hybrid systems receive 1 for both components. Purely nominal or purely structural systems receive 0.5. The evaluation is performed at the language specification level by analyzing typing judgments and formal rules.

2) Variance Formalization Score $S_{var}(L)$

$$S_{var}(L) = \frac{V_{exp}(L) + V_{sound}(L)}{2} \quad (3.3)$$

where:

- $V_{exp}(L) \in [0, 1]$ measures the explicitness of variance annotations (0 = no generics; 0.5 = implicit or restricted variance; 1 = explicit declaration of co/contra/invariance).
- $V_{sound}(L) \in [0, 1]$ measures formal soundness guarantees (1 if variance rules are formally defined and proven type-safe in specification; 0.5 if partially restricted; 0 if unsound or unspecified).

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

$$S_{rel}(L) = 1 - \frac{|E_{inh}(L) \cap E_{sub}(L)|}{|E_{sub}(L)|} \quad (3.4)$$

where:

- $E_{inh}(L)$ is the set of inheritance relations defined syntactically in the language specification.
- $E_{sub}(L)$ is the set of subtype relations defined in the language formal type system.

This metric measures the proportion of subtype relationships that are not caused by implementation inheritance. If all subtypes exist in the language only through inheritance, then the value of the metric approaches 0, which indicates the absence of conceptual separation. If subtypes are determined in most cases separately from inheritance (e.g., using interfaces or structural typing rules),

the metric approaches 1, reflecting a strong semantic separation between reuse and interchangeability. Measurement is performed at the specification level by constructing abstract inheritance and subtype relation graphs from the formal language definition.

Justification

This formula describes the differences between subtyping and inheritance mechanisms. Decomposition identifies three independent semantic aspects: the discipline of typing, variance safety, and the formal separation of inheritance from subtyping. Normalisation of coefficients allows you to adapt the expression for specific goals, increasing values of the important parameters during a specific comparison.

3.1.2 Abstraction and Encapsulation Mechanisms

Metric Definition

Let the *Abstraction and Encapsulation score (AE)* be defined as:

$$AE(L) = \alpha \cdot V(L) + \beta \cdot M(L) + \gamma \cdot I(L) + \delta \cdot E(L) \quad (3.5)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma + \delta = 1$ and each component is normalized to the interval $[0, 1]$.

Components

- $V(L)$ — *Visibility Granularity Index*. Normalized measure of the access control strictness modifiers (e.g., private, protected, file, module-level).

$$V(L) = \frac{n_{levels}(L)}{n_{max}}$$

where $n_{levels}(L)$ is the number of semantically distinct access scopes supported by the language.

- $M(L)$ — *Module Isolation Strength*. Metric aimed for comparison of the number of module or namespace boundaries that restrict access to a view:

$$M(L) = \frac{b_{strict}(L)}{b_{total}(L)}$$

where $b_{strict}(L)$ is the amount of boundaries with enforced export control (e.g., modules with explicit export lists), and $b_{total}(L)$ is the total number of modularization constructs.

- $I(L)$ — *Interface Abstraction Capacity*. Measures the ability to separate specification from implementation (e.g., interfaces, abstract classes, protocols from classes).

$$I(L) = \frac{n_{abstraction_constructs}(L)}{n_{object}(L)}$$

where $n_{abstraction_constructs}(L)$ counts distinct language-level constructs dedicated to behavioral abstraction and $n_{object}(L)$ is the number of object-defining constructs.

- $E(L)$ — *Encapsulation Enforcement Coefficient*. Binary or graded measure reflecting whether encapsulation violations are prevented by the type system and runtime (no unrestricted reflection, no unchecked field access, controlled unsafe mechanisms).

$$E(L) = 1 - \frac{n_{\text{bypass-mechanisms}}(L)}{n_{\text{total-control-mechanisms}}(L)}$$

3.1.3 Inheritance Semantics and Hierarchy Management

Metric Definition

Let the *Inheritance Semantics Robustness Index (ISRI)* be defined as:

$$ISRI(L) = \alpha \cdot H_s(L) + \beta \cdot L_d(L) + \gamma \cdot C_p(L) + \delta \cdot F_b(L), \quad (3.6)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma + \delta = 1$ and each component is normalized to the interval $[0, 1]$.

Components

- $H_s(L)$ — *Hierarchy Simplicity Factor*. A normalized measure of inheritance model complexity:

$$H_s(L) = 1 - \frac{M(L)-1}{M_{\max}-1},$$

where $M(L)$ is the maximum number of direct superclass declarations allowed (1 for single inheritance, > 1 for multiple inheritance), and $M_{\max}$ is a maximal value of $M(L)$ for all comparing languages.

- $L_d(L)$ — *Linearization Determinism Score*. A binary or graded measure reflecting whether the language defines a formally specified, deterministic method resolution order (MRO):

$$L_d(L) = \begin{cases} 1, & \text{if linearization is formally defined and deterministic,} \\ 0.5, & \text{if partially specified,} \\ 0, & \text{if ambiguous or implementation-defined.} \end{cases}$$

- $C_p(L)$ — *Constraint Protection Coefficient*. Measures availability of hierarchy restriction mechanisms:

$$C_p(L) = \frac{F(L)}{F_{max}},$$

where $F(L)$ is the number of supported protection mechanisms (e.g., final, sealed, non-virtual-by-default methods, explicit override requirements), and F_{max} is a maximal value of $M(L)$ for all comparing languages.

- $F_b(L)$ — *Fragile Base Mitigation Factor*. Reflects language-level safeguards against fragile base class effects:

$$F_b(L) = \frac{S(L)}{S_{max}},$$

where $S(L)$ counts semantic safeguards (e.g., explicit override annotations, method visibility control, requirement of explicit super-calls, covariance restrictions).

3.1.4 Composition and Alternatives to Inheritance

Metric Definition

Let the *Composition and Reuse Flexibility Index (CRFI)* be defined as:

$$CRFI(L) = \alpha \cdot D_{cls}(L) + \beta \cdot T_{exp}(L) + \gamma \cdot I_{comp}(L) \quad (3.7)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Class-Based Delegation Score $D_{cls}(L)$

$$D_{cls}(L) = \frac{K_{del}(L) + S_{fwd}(L)}{2} \quad (3.8)$$

where:

- $K_{del}(L)$ indicates available syntactic delegation constructs in the language.

$$K_{del}(L) = \begin{cases} 1, & \text{if available with keywords (e.g. delegate),} \\ 0.5, & \text{if emulated via adapters,} \\ 0, & \text{if language does not provide delegation mechanisms.} \end{cases}$$

- $S_{fwd}(L)$ evaluates the presence of mechanisms that allows to automatically delegate method calls from one object to another without writing boilerplate

code.

$$S_{fwd}(L) = \begin{cases} 1, & \text{if automatic delegation mechanisms are available,} \\ 0, & \text{if language does not provide automatic delegation.} \end{cases}$$

2) Trait Expressiveness Score $T_{exp}(L)$

$$T_{exp}(L) = \frac{R_{conf}(L) + O_{mod}(L)}{2} \quad (3.9)$$

where:

- $R_{conf}(L)$ evaluates name conflict resolution during mixing.

$$R_{conf}(L) = \begin{cases} 1, & \text{if resolution using explicit exclusion or aliasing,} \\ 0.5, & \text{if resolution using declaration order,} \\ 0, & \text{if compilation error.} \end{cases}$$

- $O_{mod}(L)$ evaluates the ability of composition mechanisms (tracks, mixins) to determine the requirements for the class that uses them.

$$O_{mod}(L) = \begin{cases} 1, & \text{if allows to declare requirements inside traits/mixins,} \\ 0, & \text{if traits/mixins should be completely self-constrained.} \end{cases}$$

3) Interface Composition Capacity $I_{comp}(L)$

$$I_{comp}(L) = \frac{N_{impl}(L) + D_{def}(L) \cdot N_{max}}{2 \cdot N_{max}} \quad (3.10)$$

where:

- $N_{impl}(L)$ is the maximum number of interfaces a class can implement,
- N_{max} is the maximum $N_{impl}(L)$ for all comparison languages.
- $D_{def}(L)$ indicates the presence of default method implementations in interfaces.

$$D_{def}(L) = \begin{cases} 1, & \text{if allows to declare implementations inside interfaces.}, \\ 0, & \text{if interfaces contain only method signatures, constant, types.} \end{cases}$$

This parameter evaluates whether interfaces function solely as types or also as units of composable behavior, approaching the role of mixins.

3.1.5 Object Representation and Creation Model

Metric Definition

Let the *Object Representation and Creation Index (ORCI)* be defined as:

$$ORCI(L) = \alpha \cdot S_{id}(L) + \beta \cdot S_{cr}(L) + \gamma \cdot S_{meta}(L) \quad (3.11)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Identity Semantics Score $S_{id}(L)$

Object identity is a core property distinguishing objects from values. This component measures the consistency of reference semantics across the language's type system.

$$S_{id}(L) = \frac{T_{ref}(L)}{T_{user}(L)} \quad (3.12)$$

where:

- $T_{user}(L)$ is the total number of user-definable type categories in the language specification.
- $T_{ref}(L)$ is the number of user-definable type categories that enforce reference semantics.

2) Instantiation Paradigm Diversity $S_{cr}(L)$

$$S_{cr}(L) = \frac{P_{supported}(L)}{P_{max}} \quad (3.13)$$

where:

- $P_{supported}(L)$ is the number of distinct instantiation paradigms supported natively (e.g., new Class, Prototype Clone, Literal Construction, Actor Spawn).
- P_{max} is the maximum number of paradigms observed across all compared languages.

3) Meta-level Extensibility Score $S_{meta}(L)$

$$S_{meta}(L) = \frac{I_{level}(L)}{I_{max}} \quad (3.14)$$

where:

- $I_{level}(L)$ represents the depth of intercession allowed (0 = none, 1 = introspection only, 2 = structural modification, 3 = creation protocol override).

$$I_{level}(L) = \begin{cases} 3, & \text{if allows overriding the object instantiation process via metaobject} \\ 2, & \text{if allows modification of existing object structures at runtime,} \\ 1, & \text{if allows querying object structure at runtime but prohibits modification} \\ 0, & \text{if does not allow querying object structure and modification.} \end{cases}$$

- I_{max} is the maximum intercession level defined for the comparison languages.

3.1.6 Polymorphism and Dispatch Mechanisms

Metric Definition

Let the *Polymorphism and Dispatch Efficiency Index (PDEI)* of a language L be defined as:

$$PDEI(L) = \alpha \cdot M_{scope}(L) + \beta \cdot S_{disp}(L) + \gamma \cdot O_{virt}(L) \quad (3.15)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Dispatch Scope Mechanism $M_{scope}(L)$

This component evaluates the richness of dispatch mechanisms supported by the language specification, distinguishing between single, multiple, and predicate

dispatch.

$$M_{scope}(L) = \frac{w_{single} \cdot I_{single} + w_{multi} \cdot I_{multi} + w_{pred} \cdot I_{pred}}{w_{single} + w_{multi} + w_{pred}} \quad (3.16)$$

where:

- $I_{single} = 1$ if single dispatch (receiver-based) is supported; 0 otherwise.
- $I_{multi} = 1$ if multiple dispatch (argument-based) is natively supported; 0 otherwise.
- $I_{pred} = 1$ if predicate-based or condition-based dispatch is available; 0 otherwise.
- Weights are set as $w_{single} = 1$, $w_{multi} = 2$, $w_{pred} = 2$
 - Single dispatch weight is 1 because it often requires structural workarounds.
 - Multiple dispatch weight is 2 because it resolves the expression problem for operation extension without modifying existing ones.
 - Predicate dispatch weight is 2 since it extends polymorphism beyond type hierarchies into execution logic.

2) Dispatch Safety Guarantee $S_{disp}(L)$

$$S_{disp}(L) = 1 - \frac{N_{runtime_error}}{N_{total_dispatch}} \quad (3.17)$$

where:

- $N_{runtime_error}$ is the number of dispatch scenarios resulting in runtime exceptions.

- $N_{total_dispatch}$ is the total amount of valid dispatch constructs defined in the language.

3) Virtualization Optimization Potential $O_{virt}(L)$

$$O_{virt}(L) = \frac{K_{final} + K_{sealed} + K_{static}}{3} \quad (3.18)$$

where:

- $K_{final} = 1$ if language support final/non-virtual methods; 0 otherwise.
- $K_{sealed} = 1$ if language support sealed class hierarchies restricting subclassing; 0 otherwise.
- $K_{static} = 1$ if language support static dispatch hints (e.g., static methods, explicit devirtualization annotations); 0 otherwise.

3.1.7 State Model, Mutability, and Concurrency Guarantees

Metric Definition

Let the *State and Concurrency Safety Index (SCSI)* be defined as:

$$SCSI(L) = \alpha \cdot M_{def}(L) + \beta \cdot A_{ctrl}(L) + \gamma \cdot P_{conc}(L) \quad (3.19)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Default Mutability Score $M_{def}(L)$

$$M_{def}(L) = \begin{cases} 1, & \text{if objects are immutable by default} \\ 0.5, & \text{if mutability requires explicit annotation} \\ 0, & \text{if objects are mutable by default} \end{cases} \quad (3.20)$$

2) Alias Control Score $A_{ctrl}(L)$

$$A_{ctrl}(L) = \frac{N_{own}(L)}{N_{max}} \quad (3.21)$$

where:

- $N_{own}(L)$ is the number of distinct ownership or linearity mechanisms supported.
- N_{max} is the maximum number of mechanisms observed across all compared languages.

3) Concurrency Primitive Safety $P_{conc}(L)$

$$P_{conc}(L) = \frac{S_{race}(L) + S_{model}(L)}{2} \quad (3.22)$$

where:

- $S_{race}(L) \in [0, 1]$ measures data race freedom guarantees.

$$S_{race}(L) = \begin{cases} 1, & \text{if the type system proves data race freedom} \\ 0.5, & \text{if runtime checks exist} \\ 0, & \text{if reliance is purely on programmer discipline} \end{cases}$$

- $S_{model}(L) \in [0, 1]$ measures the abstraction level of concurrency primitives.

$$S_{model}(L) = \begin{cases} 1, & \text{if provide high-level models(Actors, CSP channels)} \\ 0.5, & \text{if provide structured concurrency (tasks/futures)} \\ 0, & \text{if provide only low-level threads and locks} \end{cases}$$

3.1.8 Support for Formal Specification and Verifiability

Metric Definition

Let the *Formal Specification and Verifiability Index (FSVI)* be defined as:

$$FSVI(L) = \alpha \cdot C_{nat}(L) + \beta \cdot V_{enf}(L) + \gamma \cdot S_{dec}(L) \quad (3.23)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Native Contract Constructs Score $C_{nat}(L)$

$$C_{nat}(L) = \frac{n_{pre} + n_{post} + n_{inv}}{3} \quad (3.24)$$

- n_{pre} indicate the presence of first-class language constructs for preconditions.
- n_{post} indicate the presence of first-class language constructs for postconditions.
- n_{inv} indicate the presence of first-class language constructs for invariants.

- Values are assigned based on the language specification: 1 if the construct is syntactically native and semantically defined, 0 if reliant on external libraries or comments.

2) Verification Enforcement Level $V_{enf}(L)$

$$V_{enf}(L) = \begin{cases} 1.0, & \text{if static verification is supported (compile-time proof)} \\ 0.5, & \text{if only runtime checking is available} \\ 0.0, & \text{if no enforcement mechanism exists} \end{cases}$$

3) Specification-Implementation Decoupling $S_{dec}(L)$

$$S_{dec}(L) = 1 - \frac{|I_{impl} \cap I_{spec}|}{|I_{spec}|} \quad (3.25)$$

where

- I_{spec} is the set of interfaces or modules allowing independent specification.
- I_{impl} is the set of constructs where specification is tightly bound to implementation code.

Chapter 4

Implementation

The preceding chapters of this dissertation presented a comparative analysis of object models in contemporary programming languages, including C#, C++, Go, Java, Python, Ruby, JavaScript, Scala, Smalltalk, and Zonnon. The analysis was conducted according to a uniform set of formally motivated criteria covering type systems, subtyping relations, abstraction and encapsulation mechanisms, inheritance semantics, composition facilities, object representation models, dispatch strategies, state and mutability control, concurrency guarantees, and support for formal specification.

The results of that analysis demonstrate that no existing object model simultaneously achieves semantic rigor, modular robustness, resistance to hierarchical fragility, formal substitutability guarantees, and architectural scalability under concurrency constraints. These limitations are not incidental; rather, they stem from foundational design decisions concerning type theory, behavioral subtyping, and object semantics. Therefore, a purely empirical comparison is not sufficient. A constructive synthesis grounded in formal theory is required.

This section introduces the implementation chapter, whose objective is to

formulate a conceptual object model for an “ideal” programming language that systematically addresses the deficiencies identified in the comparative study. The term “ideal” is used here in a methodological sense: it denotes a theoretically consistent model derived from formal principles and evaluated against the same criteria applied to existing languages. The chapter does not propose a production-ready language, but rather a rigorously structured semantic framework.

A. Dimensions of Model Description

The object model will be described from eight principal perspectives, each reflecting a structural aspect of object-oriented language design.

1 Type-Theoretic Perspective This section includes the definition of type identity, subtype relations, compatibility rules, and variance constraints. The description will clarify how types are declared, subtype relationships are established, and type safety is enforced. Particular attention will be given to the separation between subtyping and inheritance, and to the role of contracts in determining compatibility. From this perspective, the model is treated as a formal compatibility framework that defines the semantic boundaries of programs.

2 Abstraction and Encapsulation Perspective The model will be described with respect to its abstraction boundaries. This includes the mechanisms that separate interface from implementation, the visibility rules governing program components, and the modular structure of the language. The description will specify how representation independence is preserved, how internal state is protected from external interference, and how module-level isolation is enforced. The model will therefore be examined as a system of controlled information flow.

3 Hierarchical Structure Perspective The model will be described in terms of its hierarchical organization. This perspective focuses on inheritance semantics, extension rules, and hierarchy management mechanisms.

The description will clarify:

- under which conditions inheritance is permitted,
- how method overriding is constrained,
- how further extension can be restricted,
- and how structural fragility is prevented.

From this viewpoint, the object model is analyzed as a structured extension system whose stability depends on well-defined constraints.

4 Composition and Reuse Perspective The model will be described in terms of composition and reuse. From this point of view, we consider delegation mechanisms, feature-like constructs, interface aggregation, and other non-hierarchical reuse strategies. The description will indicate how the components are combined, the possibilities of reusing behavior without exposing implementation details, and how reuse mechanisms interact with the subtype relation.

5 Object Representation and Construction Perspective The model will be described in terms of object representation and creation semantics: definition of object identity, initialization process, invariant establishment during construction, and separation between abstract type and concrete representation. The description will explain how object instances are created, how memory management and identity are supposed to be implemented at the language level.

6 Behavioral and Dispatch Perspective The model will be described from the perspective of behavioral execution and method dispatch. This includes static and dynamic dispatch rules, resolution order, overriding constraints, and interaction between polymorphism and contracts. The description will specify how behavioral substitutability is maintained at runtime and how dispatch mechanisms affect predictability and analyzability of programs.

7 State and Concurrency Perspective The model will be described with respect to state management and mutability control. This perspective examines:

- default mutability policies,
- explicit declaration of mutable entities,
- ownership or confinement rules,
- and guarantees relevant to concurrent execution.

8 Specification and Verification Perspective The model will be described from the standpoint of formal specification and verifiability. This includes the integration of invariants, preconditions, and postconditions into type definitions, as well as rules governing contract inheritance and compatibility. The description will clarify how specifications are declared, how they are checked, and how they influence subtype relations.

B. Integrative Character of the Description

In the previous part, each point of view is presented separately. To present the overall picture, this chapter highlights their interdependence. The type system

restricts inheritance; encapsulation affects composition; mutability policies affect interchangeability; and contracts determine subtype compatibility. In this way, the object model will be described as an integrated system in which each dimension contributes to the overall semantic consistency.

C. Organizational Principle

Each subsection of the chapter follows a uniform descriptive pattern:

1. Identification of the structural dimension under consideration.
2. Definition of the design objectives for that dimension.
3. Specification of the mechanisms introduced in the model.
4. Explanation of how these mechanisms interact with other dimensions.

At the first stage, the exact semantic coverage of the subsection is established. Each aspect is considered as an independently analyzed component of the overall model. Its conceptual boundaries are clearly delineated to prevent conceptual duplication and terminological ambiguity.

At the second stage, the constraints that the model must satisfy within this dimension are formalized. These goals follow from the shortcomings identified during the comparative analysis of existing languages presented earlier.

At the third stage, the constructive component of the description is introduced. Here, abstract goals are transformed into specific semantic rules: entry rules, inheritance restrictions, visibility rules, construction protocols, dispatch semantics, ownership rules, or contract distribution laws.

The fourth step is dedicated to system consistency. This integrative analysis serves two purposes: it prevents local optimizations that undermine global consistency, and demonstrates that the model behaves as a single semantic system rather than as a set of loosely coupled functions.

Chapter 5

Evaluation and Discussion

...

Chapter 6

Conclusion

...

Bibliography cited

- [1] L. Cardelli and P. Wegner, “On understanding types, data abstraction, and polymorphism,” *ACM Computing Surveys (CSUR)*, vol. 17, no. 4, pp. 471–523, 1985.
- [2] G. Booch, R. A. Maksimchuk, M. W. Engle, B. J. Young, J. Connallen, and K. A. Houston, “Object-oriented analysis and design with applications,” *ACM SIGSOFT software engineering notes*, vol. 33, no. 5, pp. 29–29, 2008.
- [3] B. Liskov, “Keynote address-data abstraction and hierarchy,” in *Addendum to the proceedings on Object-oriented programming systems, languages and applications (Addendum)*, 1987, pp. 17–34.
- [4] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.
- [5] B. H. Liskov and J. M. Wing, “Behavioral subtyping using invariants and constraints,” Tech. Rep., 1999.
- [6] E. Gamma, *Design patterns: elements of reusable object-oriented software*. Addison-Wesley, 1995, vol. 431.

- [7] M. Atkinson and R. Morrison, “Orthogonally persistent object systems,” *The VLDB Journal*, vol. 4, no. 3, pp. 319–401, 1995.
- [8] W. R. Cook, W. Hill, and P. S. Canning, “Inheritance is not subtyping,” in *Proceedings of the 17th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1989, pp. 125–135.
- [9] B. C. Pierce and D. N. Turner, “Simple type-theoretic foundations for object-oriented programming,” *Journal of functional programming*, vol. 4, no. 2, pp. 207–247, 1994.
- [10] L. Mikhajlov and E. Sekerinski, “A study of the fragile base class problem,” in *European Conference on Object-Oriented Programming*, Springer, 1998, pp. 355–382.
- [11] M. Hitz and B. Montazeri, *Measuring coupling and cohesion in object-oriented systems*. na, 1995.
- [12] P. Graham, *The Hundred-Year Language*, <https://www.paulgraham.com/hundred.html>, [Online; accessed 27-11-2025], Apr. 2003.
- [13] J. C. Reynolds, “Three approaches to type structure,” in *Colloquium on Trees in Algebra and Programming*, Springer, 1985, pp. 97–138.
- [14] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, “Design patterns: Abstraction and reuse of object-oriented design,” in *European conference on object-oriented programming*, Springer, 1993, pp. 406–431.
- [15] S. R. Chidamber and C. F. Kemerer, “A metrics suite for object oriented design,” *IEEE Transactions on software engineering*, vol. 20, no. 6, pp. 476–493, 1994.

- [16] P. Wegner, “Dimensions of object-based language design,” *ACM Sigplan Notices*, vol. 22, no. 12, pp. 168–182, 1987.
- [17] M. Vandeloise, “Towards a unified framework for programming paradigms: A systematic review of classification formalisms and methodological foundations,” *arXiv preprint arXiv:2508.00534*, 2025.
- [18] W. Crichton, G. Gray, and S. Krishnamurthi, “A grounded conceptual model for ownership types in rust,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA2, pp. 1224–1252, 2023.
- [19] D. J. Armstrong, “The quarks of object-oriented development,” *Communications of the ACM*, vol. 49, no. 2, pp. 123–128, 2006.
- [20] L. Prechelt, “An empirical comparison of seven programming languages,” *Computer*, vol. 33, no. 10, pp. 23–29, 2002.
- [21] M. S. Farooq et al., “Comparative analysis of widely use object-oriented languages,” *arXiv preprint arXiv:2306.01819*, 2023.
- [22] R. Pike, “Go at google: Language design in the service of software engineering. online,” *URL: <https://talks.golang.org/2012/splash.article>*, 2012.
- [23] I. Balbaert, *The way to Go: A thorough introduction to the Go programming language*. IUniverse, 2012.
- [24] M. Odersky and T. Rompf, “Unifying functional and object-oriented programming with scala,” *Communications of the ACM*, vol. 57, no. 4, pp. 76–86, 2014.
- [25] J. Gutknecht and E. Zueff, “Zonnon language report,” *Zurich, Institute of Computer Systems ETH Zentrum*, 2004.

- [26] R. W. Sebesta, *Concepts of programming languages*. Pearson Education India, 2016.
- [27] *Iso/iec 25010: Systems and software engineering – system and software product quality models*, 2023.